

Structured Training Program: Advance C Programming

Week 1 – Refresh on C, Pointers and Memory Management

Topic 1: Structures

C program that represents a calendar for a week. Every day has:

- dayName (eg., "Monday")
- tasks (array of string with maximum 3 tasks per day)

Note:

1. Define appropriate structures.
2. Allow the user to input taska for any day.
3. Display all taks grouped by the day.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Day {
    char dayName [10];
    char tasks [3][100];
} Day;

int main() {
    Day days[7];
    for (int i = 0; i < 7; i++) {
        days[i].dayName[0] = '\0';
        for (int j = 0; j < 3; j++) {
            days[i].tasks[j][0] = '\0';
        }
    }
    while(1) {
        char input[10];
        printf("Enter a day of the week (or 'exit' to quit): ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0) {
            break;
        }
        int index = -1;
        if (strcmp(input, "Monday") == 0) index = 0;
        else if (strcmp(input, "Tuesday") == 0) index = 1;
        else if (strcmp(input, "Wednesday") == 0) index = 2;
        else if (strcmp(input, "Thursday") == 0) index = 3;
        else if (strcmp(input, "Friday") == 0) index = 4;
        else if (strcmp(input, "Saturday") == 0) index = 5;
        else if (strcmp(input, "Sunday") == 0) index = 6;
```

```

if (index != -1) {
    strcpy(days[index].dayName, input);
    taskInput:
    printf("Enter the number of tasks for %s: ", input);
    int numTasks;
    scanf("%d%c", &numTasks);

    if(numTasks > 3){
        fprintf(stderr, "No. of tasks is greater than 3, give tasks less than
3\n");
        goto taskInput;
    }
    while(numTasks-- > 0) {
        char task[100];
        printf("Enter a task: ");
        fgets(task, sizeof(task), stdin);
        task[strcspn(task, "\n")] = '\0';
        if(strlen(task) >= 100){
            fprintf(stderr, "Task name is too long, give task name less than 100
characters\n");
            numTasks++;
            continue;
        }
        strcpy(days[index].tasks[numTasks], task);
    }
}
}
printf("\nWeekly Schedule:\n");
for (int i = 0; i < 7; i++) {
    if (strlen(days[i].dayName) > 0) {
        printf("%s:\n", days[i].dayName);
        for (int j = 0; j < 3; j++) {
            if (strlen(days[i].tasks[j]) > 0) {
                printf("  - %s\n", days[i].tasks[j]);
            }
        }
    }
}
}
}

```

Output:

```

=====
Hello Mystery Chap
=====

"Keep calm and push to main."

=====
Today: 13-05-2026   Time: 15:46:49.18
User: Mystery Chap
=====

D:\embedUR - Training\embedUR - assignments\advanced C-assessment>cd "module 1"

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>gcc structure.c -o structure.exe

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>structure.exe
Enter a day of the week (or 'exit' to quit): Thursday
Enter the number of tasks for Thursday: 1
Enter a task: Learn advanced C
Enter a day of the week (or 'exit' to quit): Friday
Enter the number of tasks for Friday: 2
Enter a task: Play Cricket with friends
Enter a task: Learn sparsity-accelerated training and q-sparse
Enter a day of the week (or 'exit' to quit): exit

Weekly Schedule:
Thursday:
- Learn advanced C
Friday:
- Learn sparsity-accelerated training and q-sparse
- Play Cricket with friends

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>

```

Topic 2: Pointers

Write a function in C that takes a pointer to an integer array and its size, and then rearranges the array in-place such that all even numbers appear before odd numbers, preserving the original relative order using only pointer arithmetics (no indexing with [])

Code:

```

#include <stdio.h>
#include <stdlib.h>

void rearrangeArray(int* arr, int size) {
    for (int i = 1; i < size; i++) {
        if (*(arr + i) % 2 == 0) {
            int key = *(arr + i);
            int j = i - 1;
            while (j >= 0 && *(arr + j) % 2 != 0) {
                *(arr + j + 1) = *(arr + j);
                j--;
            }
            *(arr + j + 1) = key;
        }
    }
}

int main() {
    int size;
    printf("Enter the size of the array: ");
    scanf("%d", &size);
    int* arr = (int*)malloc(size * sizeof(int));

```

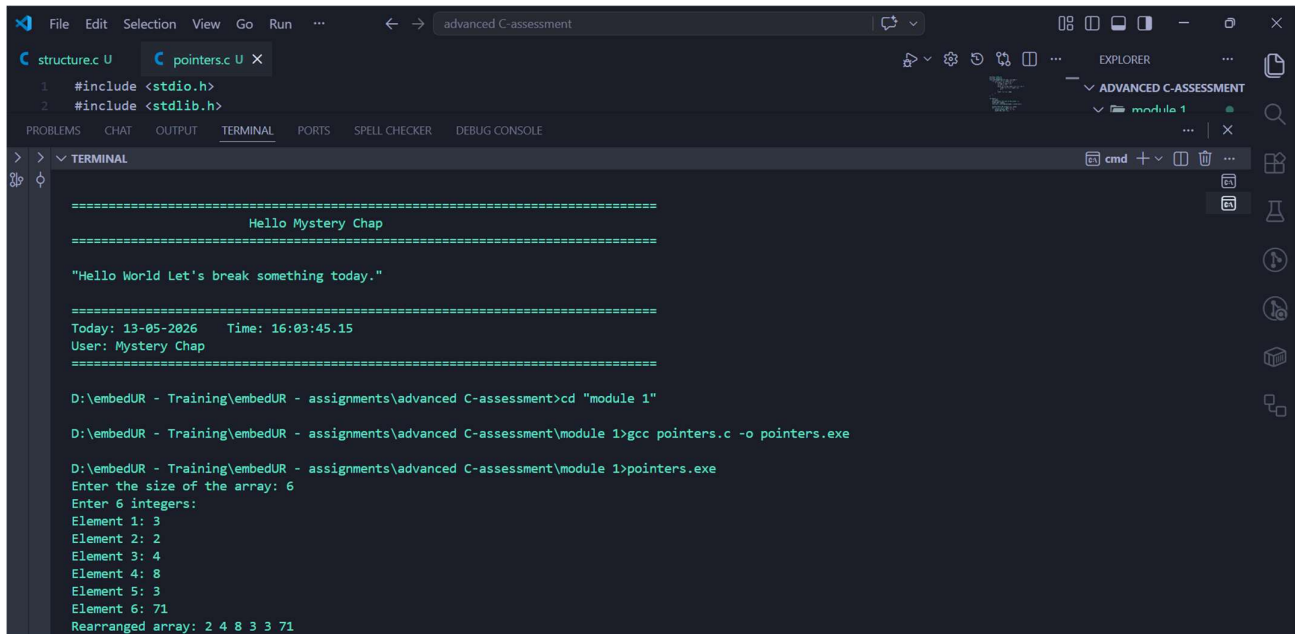
```
printf("Enter %d integers:\n", size);
for (int i = 0; i < size; i++) {
    printf("Element %d: ", i + 1);
    scanf("%d", arr + i);
}

rearrangeArray(arr, size);

printf("Rearranged array: ");
for (int i = 0; i < size; i++) {
    printf("%d ", *(arr + i));
}
printf("\n");
free(arr);

return 0;
}
```

Output:



```
File Edit Selection View Go Run ... advanced C-assessment
structure.c U pointers.c U X
1 #include <stdio.h>
2 #include <stdlib.h>
PROBLEMS CHAT OUTPUT TERMINAL PORTS SPELL CHECKER DEBUG CONSOLE
> >
=====
Hello Mystery Chap
=====

"Hello World Let's break something today."

=====
Today: 13-05-2026 Time: 16:03:45.15
User: Mystery Chap
=====

D:\embedUR - Training\embedUR - assignments\advanced C-assessment>cd "module 1"

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>gcc pointers.c -o pointers.exe

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>pointers.exe
Enter the size of the array: 6
Enter 6 integers:
Element 1: 3
Element 2: 2
Element 3: 4
Element 4: 8
Element 5: 3
Element 6: 71
Rearranged array: 2 4 8 3 3 71
```

Topic 3: Arrays

You are given a 2D matrix of size $n \times n$ where each row and each column is sorted in increasing order. Write a C function to determine whether the given key exists in the matrix using the most efficient approach.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int keyExistInMatrix(int** matrix, int rows, int cols, int key) {
    int x = 0, y = cols - 1;
    while (x < rows && y >= 0) {
        if (matrix[x][y] == key)
            return 1;
        else if (matrix[x][y] > key)
            y--;
        else
            x++;
    }
    return 0;
}

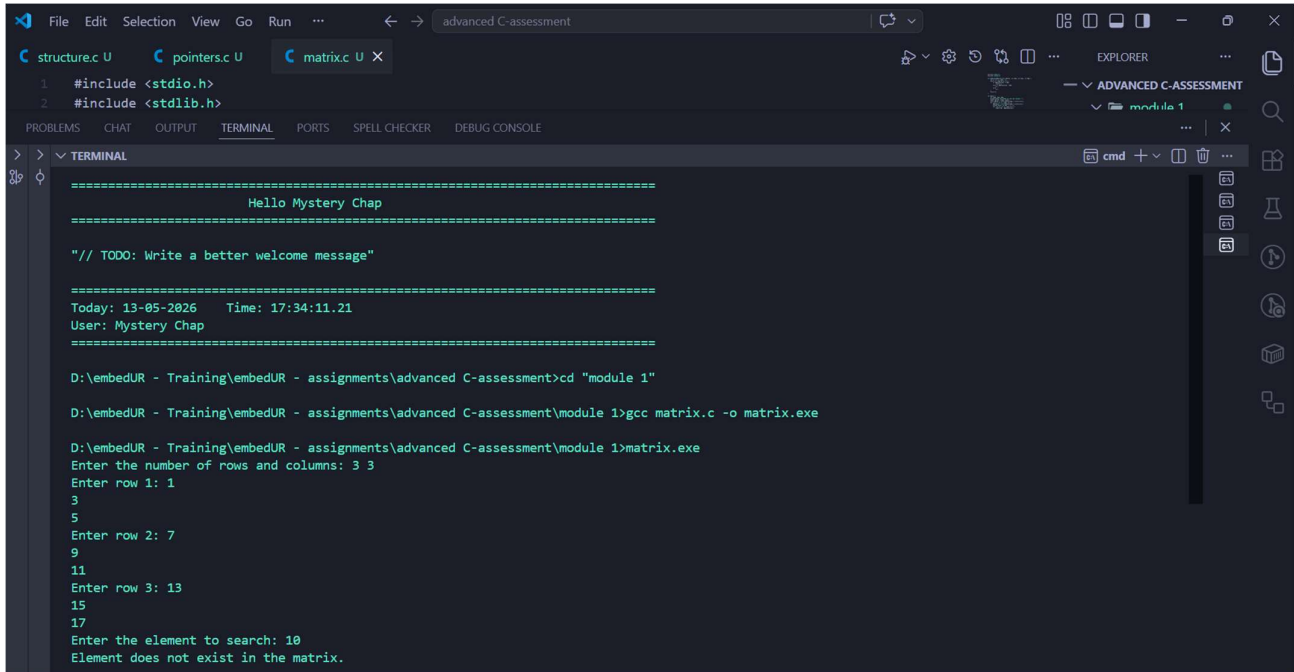
int main() {
    int rows, cols, key;
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);
    int** matrix = (int**)malloc(rows * sizeof(int*));
    for (int i = 0; i < rows; i++) {
        matrix[i] = (int*)malloc(cols * sizeof(int));
        printf("Enter row %d: ", i + 1);
        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
    printf("Enter the element to search: ");
    scanf("%d", &key);

    if (keyExistInMatrix(matrix, rows, cols, key))
        printf("Element exists in the matrix.\n");
    else
        printf("Element does not exist in the matrix.\n");

    for (int i = 0; i < rows; i++)
        free(matrix[i]);

    free(matrix);
    return 0;
}
```

Output:

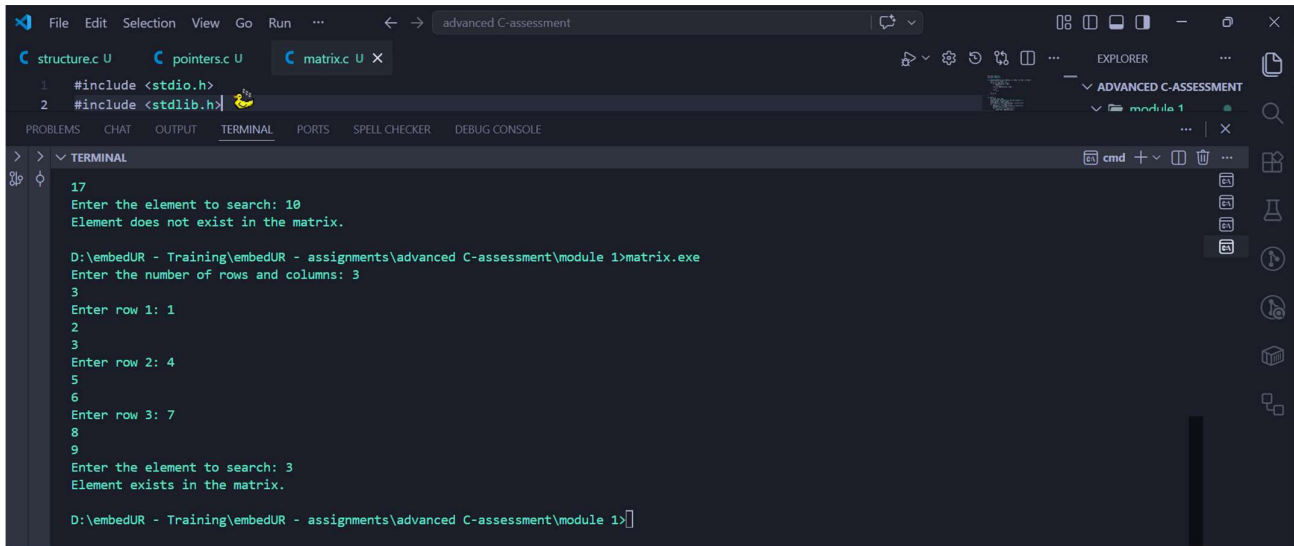


```
File Edit Selection View Go Run ... advanced C-assessment
structure.c U pointers.c U matrix.c U X
1 #include <stdio.h>
2 #include <stdlib.h>
PROBLEMS CHAT OUTPUT TERMINAL PORTS SPELL CHECKER DEBUG CONSOLE
> > TERMINAL cmd +v
=====
Hello Mystery Chap
=====
"// TODO: Write a better welcome message"
=====
Today: 13-05-2026 Time: 17:34:11.21
User: Mystery Chap
=====

D:\embedUR - Training\embedUR - assignments\advanced C-assessment>cd "module 1"

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>gcc matrix.c -o matrix.exe

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>matrix.exe
Enter the number of rows and columns: 3 3
Enter row 1: 1
3
5
Enter row 2: 7
9
11
Enter row 3: 13
15
17
Enter the element to search: 10
Element does not exist in the matrix.
```



```
17
Enter the element to search: 10
Element does not exist in the matrix.

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>matrix.exe
Enter the number of rows and columns: 3
3
Enter row 1: 1
2
3
Enter row 2: 4
5
6
Enter row 3: 7
8
9
Enter the element to search: 3
Element exists in the matrix.

D:\embedUR - Training\embedUR - assignments\advanced C-assessment\module 1>
```